

Development Tools for Machine Learning and AI

Aldebaro Klautau
Federal University of Pará (UFPA) / LASSE

Computational Intelligence Class

Python, environments, Jupyter notebooks and virtual machines

August 27, 2026

Increase your empregability by learning the following for this class:

- § **1. Use a terminal (or console) to execute Python code and Jupyter notebooks from the command line.**
 - § Learn about the PATH and PYTHONPATH environment variables.
- § **2. Create and manage your own Python environment with a specified Python version.**
 - § Learn to create, manage, and reproduce **Python environments** using tools such as venv, Miniconda, and Conda.
- § **3. Become familiar with Visual Studio Code (VS Code).**
 - § Activate and use a specified Python environment within VS Code.
 - § Use a given AI model to assist with programming.
 - § Debug Python code using breakpoints.

Learn the following tools for this class (2):

- **4. Develop Jupyter notebooks and execute them in the cloud using Google Colab, Kaggle, or a similar service.**
 - Add a “**Open in Colab**” badge so that your notebook can be opened directly in Google Colab.
- **5. Learn Git and use GitHub for version control and collaborative development.**
 - Create and manage a repository.
 - Work with **issues, forks, and pull requests**.
 - Host a website on GitHub using **GitHub Pages** (e.g., <https://<username>.github.io>).
 - Use **GitHub Actions** for CI/CD and automatically deploy a page whenever changes are committed.

Learn the following tools for this class (3):

- **6. Learn about Kaggle and participate in a Kaggle competition.**
 - Learn how to explore Kaggle datasets, notebooks, and competitions.
 - Develop and submit a solution to a Kaggle competition.
 - Document your approach and results so that others can reproduce your work.
- **7. Learn how to use third-party virtual machines and Docker, and how to create and manage your own virtual machines and Docker containers.**
- **8. Develop software that enables reproducible results by organizing and packaging the software so that it can be easily used by other engineers.**
 - Provide clear instructions for installation, configuration, and execution.
 - Manage dependencies and software versions.
 - Automate experiments and, when appropriate, provide scripts or notebooks that reproduce the reported results.

Learn the following tools for this class (4):

- **9. Become familiar with tools for developing and evaluating deep learning models, such as TensorBoard and Optuna.**
 - Use **TensorBoard** to monitor and visualize the training process.
 - Use **Optuna** for hyperparameter optimization.
- **10. Become familiar with foundation models and modern AI development frameworks.**
 - Learn to use **Hugging Face** for accessing, evaluating, fine-tuning, and deploying pretrained models.
 - Become familiar with important resources and platforms for AI development, such as **Hugging Face, PyTorch, TensorFlow, OpenAI, and Google Colab**.
 - Learn to use **foundation models** for tasks such as natural language processing (NLP), computer vision, speech processing, and multimodal AI.
 - Learn **transfer learning and fine-tuning**, including **freezing and unfreezing layers, selective fine-tuning**, and **parameter-efficient fine-tuning (PEFT)** methods such as **LoRA**.
 - Develop advanced AI applications combining **reinforcement learning (RL)**, **large language models (LLMs)**, and **graph neural networks (GNNs)**.

Importance of proper management of Python environments



conda list

Example:

Keras in TF2 uses hdf5 file format

conda install h5py==3.7.0

Keras folder with code that requires h5py:
d:\Programs\Anaconda3\envs\ai6g\Lib\site-packages\h5py

09/11/2022 05:25 PM 3,334,144 hdf5.dll

09/11/2022 05:25 PM 117,760 hdf5_hl.dll

Folder d:\Programs\Anaconda3\envs\ai6g\Lib\site-packages\keras\saving

09/05/2022 02:22 PM 37,436

hdf5_format.py

```
# packages in environment at d:\Programs\Anaconda3\envs\ai6g:
#
# Name                    Version            Build                Channel
absl-py                   1.2.0              pypi_0              pypi
aom                       3.4.0              h0e60522_1          conda-forge
asttokens                 2.0.8              pyhd8ed1ab_0        conda-forge
...
gym                      0.19.0             py39h832f523_0      conda-forge
h5py                     3.7.0              pypi_0              pypi
...
tensorboard              2.9.1              pypi_0              pypi
tensorboard-data-server  0.6.1              pypi_0              pypi
tensorboard-plugin-wit   1.8.1              pypi_0              pypi
tensorflow                2.9.2              pypi_0              pypi
tensorflow-estimator     2.9.0              pypi_0              pypi
tensorflow-io-gcs-filesystem 0.26.0            pypi_0              pypi
termcolor                 1.1.0              pypi_0              pypi
...
```

Introduction to Jupyter notebooks

A screenshot of a Jupyter Notebook interface in a web browser. The browser address bar shows 'localhost:4321/lab/tree/01_detection_qam_over_awgn.ipynb'. The interface includes a top menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help) and a toolbar with icons for file operations. On the left, a file browser sidebar shows a directory structure with files like 'files_01_detection', 'files_02_channel', etc., and a table of 'Last Modified' times. The main area displays the notebook content, which includes a title 'Inteligência Artificial e Aprendizado de Máquina Aplicados a Redes 5G e 6G', a subtitle 'Aldebaro Klautau (UFPA). Minicurso 5 do SBrT - 25 de setembro de 2022.', a section header 'Detection of QAM over AWGN using machine learning', and a 'Goals:' section with two numbered points. Below the goals is a code cell with a Python script to clone a repository and install dependencies. An orange arrow points to the right side of the notebook content area.

localhost:4321/lab/tree/01_detection_qam_over_awgn.ipynb

File Edit View Run Kernel Tabs Settings Help

Filter files by name

Name	Last Modified
files_01_detection	2 days ago
files_02_channel	2 days ago
files_03_channel	3 days ago
files_04_beam	3 days ago
files_05_data	4 days ago
files_05_federated	3 days ago
files_06_channel	3 days ago
files_07_end	2 days ago
files_08_mdp	3 days ago
files_09_rl	2 days ago
files_10_channel	3 days ago
files_11_time	19 hours ago
logs	3 days ago
mimo_rl	3 days ago
00_notebook_templa...	2 days ago
01_detection_qam_ov...	2 days ago
02_channel_estimatio...	2 days ago
03_channel_estimatio...	2 days ago

Open in Colab

Inteligência Artificial e Aprendizado de Máquina Aplicados a Redes 5G e 6G. Aldebaro Klautau (UFPA). Minicurso 5 do SBrT - 25 de setembro de 2022.

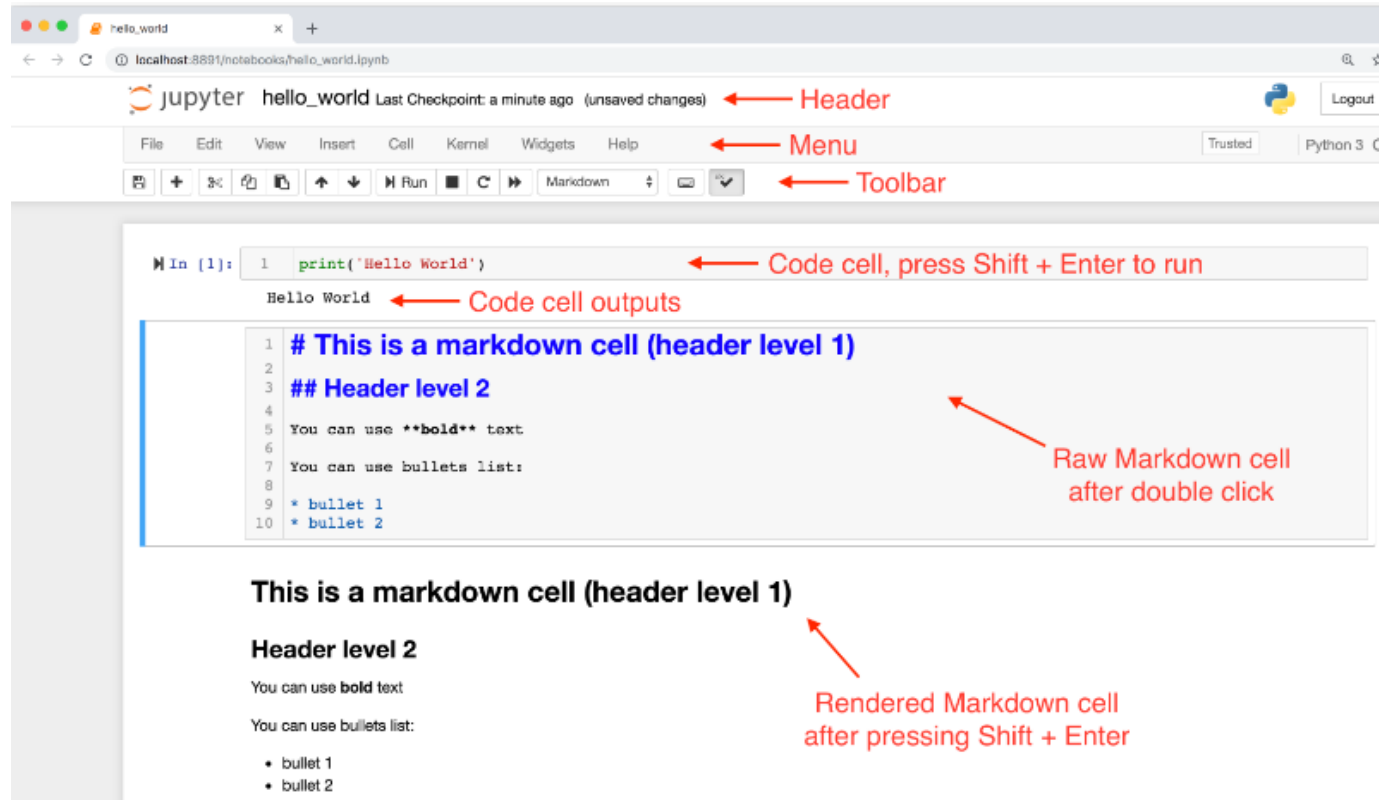
Detection of QAM over AWGN using machine learning

Goals:

1. To get results as close as possible of the optimum provided by the theoretical equation for a given signal-to-noise ratio (SNR). You can modify any value in this notebook, but keep the modulation order as $M=16$ symbols (or classes, in the context of AI/ML), such that we have 4 bits per symbol.
2. Make sure the symbol error rate (SER) estimations obtained with a test set and cross-validation are consistent.

```
[213]: # Clone the repository if running in Colab and install all the dependencies
if 'google.colab' in str(get_ipython()):
    import sys
    import os
    try:
        !git clone https://github.com/aldebaro/ai6g.git
    except:
        print("ai6g is already in the contents")
```

*Introduction to Jupyter notebooks (2)



The screenshot shows a Jupyter Notebook interface in a web browser. The browser address bar shows `localhost:8891/notebooks/hello_world.ipynb`. The Jupyter header includes the logo, the notebook name `hello_world`, and a status message `Last Checkpoint: a minute ago (unsaved changes)`. The menu bar includes `File`, `Edit`, `View`, `Insert`, `Cell`, `Kernel`, `Widgets`, and `Help`. The toolbar includes icons for `New`, `Open`, `Save`, `Run`, `Stop`, `Restart`, `Clear`, `Undo`, `Redo`, `Markdown`, `Code`, and `Raw`. The notebook content consists of three cells: a code cell, a raw markdown cell, and a rendered markdown cell. Red arrows point to various elements: the header, the menu, the toolbar, the code cell, the code cell outputs, the raw markdown cell, and the rendered markdown cell.

Annotations in the image:

- Header
- Menu
- Toolbar
- Code cell, press Shift + Enter to run
- Code cell outputs
- Raw Markdown cell after double click
- Rendered Markdown cell after pressing Shift + Enter

Code cell content:

```
In [1]: 1 print('Hello World')
```

Code cell outputs:

```
Hello World
```

Raw Markdown cell content:

```
1 # This is a markdown cell (header level 1)
2
3 ## Header level 2
4
5 You can use bold text
6
7 You can use bullets list:
8
9 * bullet 1
10 * bullet 2
```

Rendered Markdown cell content:

This is a markdown cell (header level 1)

Header level 2

You can use **bold** text

You can use bullets list:

- bullet 1
- bullet 2

*Jupyter shortcuts, magic and shell commands



Two different keyboard input modes:

- **Edit** mode: type code or text into a cell.
Green cell border
- **Command** mode: notebook level commands.
Gray cell border with a blue left margin

List of shortcuts:

Go to menu: Help → Keyboard Shortcuts

Shortcuts that work in both edit and command modes:

Shift + Enter - run the current cell, select below

Ctrl + Enter - run selected cells

Alt + Enter - run the current cell, insert below

Ctrl + S - save and checkpoint

Magic commands:

`%matplotlib inline` - **Display matplotlib graphs in notebook**

`%run <file name>` - **Run a file**

`%%time` - **Get an execution time**

`%who` - **List all variables**

`%pinfo <variable>` - **Get detailed information about variable**

`%env` - **List all environment variables**

`%load_ext autoreload` - **Reload modules**

`%pip` - **Install in current kernel (instead initially launched)**

`%conda` - **Install in current kernel (instead initial)**

Shell commands in IPython / Jupyter:

Use the `!` character as prefix to the command. For instance:

`!ls` (on Linux)

`!dir` (on Windows)

[1] <https://towardsdatascience.com/jupyter-notebook-shortcuts-bf0101a98330>
[2] <https://towardsdatascience.com/top-8-magic-commands-in-jupyter-notebook-c1582e813560>
[3] <https://jakevdp.github.io/PythonDataScienceHandbook/01.05-ipython-and-shell-commands.html>

Machine and deep learning frameworks



DL training frameworks implement sophisticated algorithms, which use “backpropagation”, automatic differentiation and stochastic gradient descent (SGD)



Most used language



Facebook's / Meta's



Google's, TF versions 1 and 2, with high level Keras API

&



Deployment frameworks: facilitate pruning the models and quantizing the weights for acceleration

Qualcomm's AI Model Efficiency Toolkit

www.tinyml.org



TorchScript, Tensorflow Lite & PyTorch Quantization



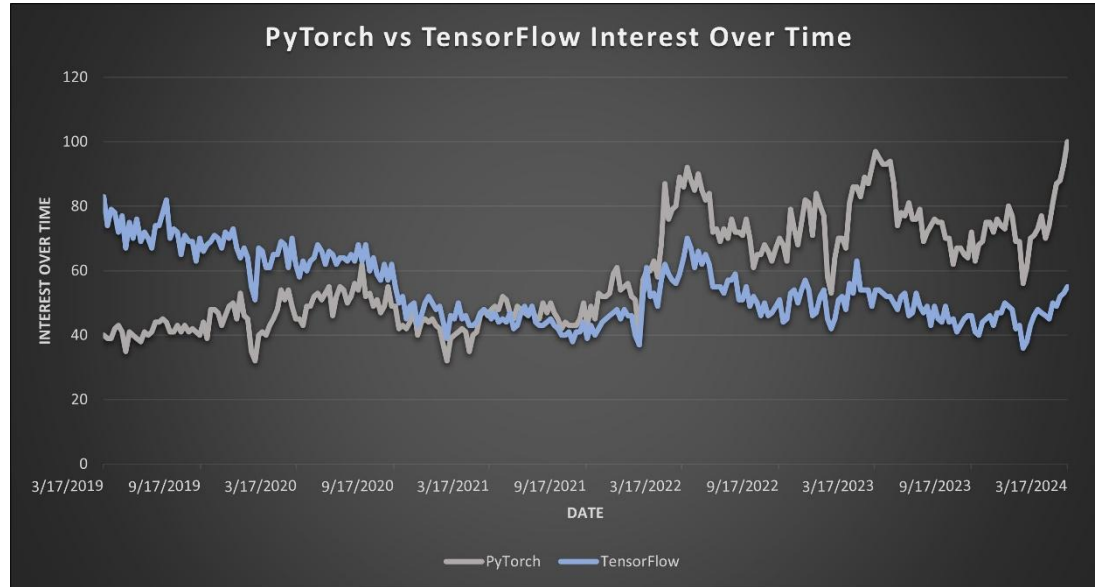
Other tools: NVIDIA, Intel, etc.

Auxiliary tools for (shallow) machine learning, debugging, assessing models and running on cloud



It may not be trivial to set up your development workflow

*Tensorflow 2 versus Pytorch



TF2: Easier deployment to cloud, servers, mobile, and IoT devices: TensorFlow Serving and TensorFlow Lite

Pytorch: More models, for instance, at <https://huggingface.co/>

- 85% only PyTorch
- 8% only TF
- 7% both

Tools such as:

<https://github.com/Lyken17/pytorch-OpCounter> to count MACs / FLOPs